

## LAB# 8

### Introduction to MySQL GROUP BY clause

The GROUP BY clause groups a set of rows into a set of summary rows by values of columns or expressions. The GROUP BY clause returns one row for each group. In other words, it reduces the number of rows in the result set.

SELECT c1, c2,..., cn, aggregate\_function(ci) FROM

table

WHERE

where\_conditions

GROUP BY c1 , c2,...,cn;

#### A) Simple MySQL GROUP BY example

Let's take a look at the orders table in the sample database.

| orders         |
|----------------|
| * orderNumber  |
| orderDate      |
| requiredDate   |
| shippedDate    |
| status         |
| comments       |
| customerNumber |

Suppose you want to group values of the order's status into subgroups, you use the GROUP BY

clause with the status column as the following query:

```
SELECT status FROM orders  
GROUP BY status;
```

|   | status     |
|---|------------|
| ▶ | Cancelled  |
|   | Disputed   |
|   | In Process |
|   | On Hold    |
|   | Resolved   |
|   | Shipped    |

As you can see clearly from the output, the GROUP BY clause returns unique occurrences of status values. It works like the DISTINCT operator as shown in the following query:

```
SELECT DISTINCT  
    status  
FROM  
    orders;
```

### Introduction to MySQL HAVING clause

The HAVING clause is used in the SELECT statement to specify filter conditions for a group of rows or aggregates.

```
SELECTselect_listFROMtable_name WHERE  
    search_conditionGROUP BYgroup_by_expression HAVING  
    group_condition;
```

## MySQL HAVING clause examples

orderdetails table in the sample database for the demonstration.

| orderdetails    |
|-----------------|
| * orderNumber   |
| * productCode   |
| quantityOrdered |
| priceEach       |
| orderLineNumber |

The following uses the GROUP BY clause to get order numbers, the number of items sold per order, and

total sales for each from the orderdetails table:

```
SELECT orderNumber, SUM(quantityOrdered) AS itemsCount,
SUM(priceEach*quantityOrdered) AS total
FROM
    orderdetails
GROUP BY orderNumber;
```

|   | orderNumber | itemsCount | total    |
|---|-------------|------------|----------|
| ▶ | 10100       | 151        | 10223.83 |
|   | 10101       | 142        | 10549.01 |
|   | 10102       | 80         | 5494.78  |
|   | 10103       | 541        | 50218.95 |
|   | 10104       | 443        | 40206.20 |
|   | 10105       | 545        | 53959.21 |
|   | 10106       | 675        | 52151.81 |
|   | 10107       | 229        | 22292.62 |

```

SELECT
    ordernumber,
    SUM(quantityOrdered) AS itemsCount,
    SUM(priceeach*quantityOrdered) AS total
FROM
    orderdetails
GROUP BY ordernumber HAVING
    total > 1000;

```

|   | ordernumber | itemsCount | total    |
|---|-------------|------------|----------|
| ► | 10100       | 151        | 10223.83 |
|   | 10101       | 142        | 10549.01 |
|   | 10102       | 80         | 5494.78  |
|   | 10103       | 541        | 50218.95 |
|   | 10104       | 443        | 40206.20 |
|   | 10105       | 545        | 53959.21 |
|   | 10106       | 675        | 52151.81 |
|   | 10107       | 229        | 22292.62 |

## Introduction to MySQL join clauses

A join is a method of linking data between one (self-join) or more tables based on values of the common column between the tables.

MySQL supports the following types of joins: Inner join Left join Right join Cross join

To join tables, you use the cross join, inner join, left join, or right join clause. The join Clause is used in the SELECT statement appeared after the FROM clause.

First, create two tables called members and committees:

```
CREATE TABLE members (  
  member_id INT AUTO_INCREMENT,  
  name VARCHAR(100),  
  PRIMARY KEY (member_id)  
);
```

```
CREATE TABLE committees (  
  committee_id INT AUTO_INCREMENT,  
  name VARCHAR(100),  
  PRIMARY KEY (committee_id)  
);
```

## MySQL INNER JOIN clause

```
SELECT column_list FROM table_1  
INNER JOIN table_2 ON join_condition;
```

```
SELECT column_list  
  
FROM table_1  
  
INNER JOIN table_2 USING (column_name);
```

The following statement uses an inner join clause to find  
members who are also the committee members:

```
SELECT m.member_id, m.name AS member, c.committee_id, c.name AS committee  
FROM  
  
    members m  
  
INNER JOIN committees c ON c.name = m.name;
```

```
+-----+-----+---+-----+          +  
| member_id | member | committee_id | committee |  
  
+-----+-----+---+-----+          +  
  
|      1 | John  |      1 | John  |  
  
|      3 | Mary  |      2 | Mary  |  
  
|      5 | Amelia |      3 | Amelia |
```

+ - - - - + - - - - + - - - - + - - - - +

3 rows in set (0.00 sec)

In this example, the inner join clause use the values in the name columns in both tables members and committees to match.